

The `BCC_nl_Eshelby` Hybrid Mobility Law Source Guide and Mathematical Formulation

KGParadis source-level technical documentation

Code snapshot reviewed 7 August 2026

Abstract

The mobility selector `BCC_nl_Eshelby` combines two existing BCC constitutive laws on each arm attached to a node. Portions outside the union of Eshelby inclusions use the nonlinear screw and linear edge/junction model from `MobilityLaw_BCC_nl.cc`; portions inside use exactly the character-dependent drag tensor from `MobilityLaw_BCC_0.cc`. This document derives the inclusion weight, both constitutive branches, their hybrid residual and Jacobian, and the corresponding source and build wiring. The implementation is a complete, independent translation unit; the original `MobilityLaw_BCC_nl.cc` is not modified.

Contents

1 Purpose and source layout	2
2 Arm geometry and notation	2
3 Eshelby coverage of an arm	2
3.1 Ellipsoid intersection and periodic images	2
3.2 Nodal rather than geometric length fraction	3
4 Outside law: original BCC nonlinear mobility	3
4.1 Temperature- and pressure-dependent screw constants	3
4.2 Screw force and linear edge/junction tensors	4
5 Inside law: exact BCC_0 arm tensor	4
6 Hybrid residual and Jacobian	5
7 Frames, outputs, and limiting cases	6
8 Parameters and control-file use	6
8.1 Complete example control input	7
8.2 Inputs specific to the hybrid switch	9
9 Build and verification	9
10 Implementation cautions	9

1 Purpose and source layout

The public entry points are

```
InitMob_BCC_nl_Eshelby(Home_t *),
Mobility_BCC_nl_Eshelby(Home_t *,Node_t *,MobArgs_t *).
```

They are implemented in `src/MobilityLaw_BCC_nl_Eshelby.cc`. The control-file selector is

```
mobilityLaw = "BCC_nl_Eshelby".
```

The source is compiled only when `ESHELBY` is defined. It contains its own copies of the private BCC nonlinear helpers and its own file-static constants `Linear`, `NonLinear`, and `V0`. Its initializer must therefore compute those constants itself; calling `InitMob_BCC_nl` would initialize a different translation unit's private state.

The implementation and registration are divided as follows.

File	Responsibility
<code>src/MobilityLaw_BCC_nl_Eshelby.cc</code>	Complete mobility implementation, inclusion interval union, BCC_0 tensor, hybrid Newton residual and Jacobian.
<code>include/Mobility.h</code>	Guarded enumeration, prototypes, and selector attributes.
<code>src/DisableUnneededParams.cc</code>	Retains the union of BCC_nl, BCC_0, pressure, and inclusion parameters in restart/control output.
<code>makefile.srccs</code> and <code>src/makefile.dep</code>	Object registration and include dependencies.

2 Arm geometry and notation

For an arm from the current node at $\mathbf{x}_1$ to a neighboring node, the minimum-image segment vector and its length are

$$\mathbf{r} = \text{ZImage}(\mathbf{x}_2 - \mathbf{x}_1), \quad L = \|\mathbf{r}\|, \quad \mathbf{t} = \frac{\mathbf{r}}{L}. \quad (1)$$

Let $\mathbf{b}$ be the arm Burgers vector. The BCC nonlinear decomposition uses the projected screw and edge lengths

$$L_s = \frac{|\mathbf{r} \cdot \mathbf{b}|}{\|\mathbf{b}\|}, \quad L_e = \sqrt{\max(0, L^2 - L_s^2)}. \quad (2)$$

Every arm contribution has the usual nodal factor 1/2. Vectors and tensors below are evaluated in the crystal frame used by the copied nonlinear solver; Section 7 describes frame conversion.

The source classifies an arm as a product-zero or junction Burgers type when

$$|b_x b_y b_z| < 10^{-12}. \quad (3)$$

Other Burgers vectors use the regular BCC $\langle 111 \rangle$ branch.

3 Eshelby coverage of an arm

3.1 Ellipsoid intersection and periodic images

For an inclusion centered at $\mathbf{C}$, with semi-axes a_1, a_2, a_3 and orthonormal principal directions $\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3$, its interior is

$$\sum_{k=1}^3 \left(\frac{\mathbf{q}_k \cdot (\mathbf{x} - \mathbf{C})}{a_k} \right)^2 \leq 1. \quad (4)$$

The code first moves $\mathbf{C}$ to the periodic image nearest $\mathbf{x}_1$. It then intersects

$$\mathbf{x}(s) = \mathbf{x}_1 + s\mathbf{r}, \quad 0 \leq s \leq 1, \quad (5)$$

with the ellipsoid by transforming it to a unit sphere. Each intersected particle produces a clamped interval $[a_k, b_k] \subseteq [0, 1]$. Tangent or zero-width intervals are discarded. The intervals are sorted and merged so overlapping inclusions are counted only once. Denote their union by U .

3.2 Nodal rather than geometric length fraction

The inside weight is not simply the covered length. For the current endpoint the linear nodal shape function is $N_1(s) = 1 - s$, so the normalized covered weight is

$$\phi = 2 \int_U (1 - s) ds = \sum_{[a_k, b_k] \subset U} \left[(1 - a_k)^2 - (1 - b_k)^2 \right]. \quad (6)$$

Thus $0 \leq \phi \leq 1$, with $\phi = 0$ for an uncovered arm and $\phi = 1$ for a wholly covered arm. Reversing the arm at its neighboring endpoint naturally changes the shape-function weight. This is why the law does not perform a binary switch based only on whether the node position is inside a particle.

If `inclusionFile` is empty, `enableInclusions` is false and the helper returns $\phi = 0$ without doing intersection work.

4 Outside law: original BCC nonlinear mobility

4.1 Temperature- and pressure-dependent screw constants

Write

$$p = \text{MobExpP}, \quad q = \text{MobExpQ}, \quad \alpha = \text{MobAlpha}, \quad H = \frac{\text{MobDeltaH0}}{k_B T},$$

and let $P_{\text{GPa}} = P/10^9$. The initializer computes

$$\mathcal{L} = AC_0 \left(1 - C \frac{T}{T_m} + DP_{\text{GPa}} \right), \quad (7)$$

$$y = \alpha^p,$$

$$\mathcal{N} = \frac{\mathcal{L}}{\exp[-H(1-y)^q] [1 + pqH(1-y)^{q-1}y]}, \quad (8)$$

$$V_0 = \alpha\mathcal{L} - \alpha\mathcal{N} \exp[-H(1-y)^q]. \quad (9)$$

Here A, C_0, C, D, T_m correspond to `MobCoeffA`, `MobCoeffC0`, `MobCoeffC`, `MobCoeffD`, and `meltTemp`. The per-Burgers-magnitude quantities used by the force law are

$$M_L = \frac{\mathcal{L}}{b_{\text{mag}}}, \quad M_N = \frac{\mathcal{N}}{b_{\text{mag}}}, \quad v_0 = \frac{V_0}{b_{\text{mag}}}. \quad (10)$$

For one screw Burgers family, define

$$f_P = \text{MobPeierls} \|\mathbf{b}\|, \quad \omega = \frac{M_N}{f_P}, \quad v_{\text{tr}} = \alpha M_L - v_0. \quad (11)$$

For $0 < v < v_{\text{tr}}$, the scalar glide force $f(v)$ is the numerical inverse of

$$v = \omega f \exp[-H(1 - (f/f_P)^p)^q]. \quad (12)$$

For $v \geq v_{\text{tr}}$, the continuation is linear,

$$f(v) = \frac{(v + v_0)f_P}{M_L}. \quad (13)$$

The source solves Equation (12) with its inherited Newton iterations and also returns $f'(v)$.

4.2 Screw force and linear edge/junction tensors

Let $\hat{\mathbf{b}} = \mathbf{b} / \|\mathbf{b}\|$, $P_b = I - \hat{\mathbf{b}} \otimes \hat{\mathbf{b}}$, and

$$\mathbf{v}_\perp = P_b \mathbf{v}, \quad v_\perp = \|\mathbf{v}_\perp\|, \quad \hat{\mathbf{v}}_\perp = \mathbf{v}_\perp / v_\perp. \quad (14)$$

The nonlinear screw force per unit screw length is

$$\mathbf{F}_s(\mathbf{v}; \mathbf{b}) = B_l \hat{\mathbf{b}} \otimes \hat{\mathbf{b}} \mathbf{v} + f(v_\perp) \hat{\mathbf{v}}_\perp, \quad B_l = \frac{1}{\text{MobLine}}. \quad (15)$$

Away from $v_\perp = 0$, its derivative is

$$D_s = B_l \hat{\mathbf{b}} \otimes \hat{\mathbf{b}} + f'(v_\perp) \hat{\mathbf{v}}_\perp \otimes \hat{\mathbf{v}}_\perp + \frac{f(v_\perp)}{v_\perp} (P_b - \hat{\mathbf{v}}_\perp \otimes \hat{\mathbf{v}}_\perp). \quad (16)$$

The implementation uses the finite zero-velocity limit of this expression.

For a regular arm, set $\mathbf{n} = (\mathbf{b} \times \mathbf{t}) / \|\mathbf{b} \times \mathbf{t}\|$ and use the orthogonal line direction $\mathbf{t}$ up to an irrelevant sign. Its linear edge tensor is

$$D_e = B_g \hat{\mathbf{b}} \otimes \hat{\mathbf{b}} + B_c \mathbf{n} \otimes \mathbf{n} + B_l \mathbf{t} \otimes \mathbf{t}, \quad B_g = \frac{1}{\text{MobGlide}}, \quad B_c = \frac{1}{\text{MobClimb}}. \quad (17)$$

For a product-zero/junction arm, the whole arm uses

$$D_j = B_c(I - \mathbf{t} \otimes \mathbf{t}) + B_l \mathbf{t} \otimes \mathbf{t}. \quad (18)$$

5 Inside law: exact BCC_0 arm tensor

The inside portion does not use a scalar Eshelby resistance multiplier. It uses the character-dependent tensor from `MobilityLaw_BCC_0.cc`. Define

$$B_s = \frac{1}{\text{MobScrew}}, \quad B_e = \frac{1}{\text{MobEdge}}, \quad B_{ec} = \frac{1}{\text{MobClimb}}, \quad B_{l0} = 10^{-2} \min(B_s, B_e), \quad (19)$$

and

$$c^2 = \frac{(\mathbf{t} \cdot \mathbf{b})^2}{\|\mathbf{b}\|^2}. \quad (20)$$

For a product-zero Burgers vector, Equation (3), the BCC_0 tensor is

$$\boxed{B^0 = B_{ec}I + (B_{l0} - B_{ec})\mathbf{t} \otimes \mathbf{t}.} \quad (21)$$

For a regular near-screw arm, $1 - c^2 \leq 10^{-12}$, it is

$$\boxed{B^0 = B_sI + (B_{l0} - B_s)\mathbf{t} \otimes \mathbf{t}.} \quad (22)$$

For a mixed regular arm, form the orthonormal directions

$$\mathbf{n} = \frac{\mathbf{b} \times \mathbf{t}}{\sqrt{(1 - c^2) \|\mathbf{b}\|^2}}, \quad \mathbf{m} = \mathbf{n} \times \mathbf{t}, \quad (23)$$

and the interpolated glide and climb drags

$$B_g^0 = \left[B_e^{-2} + (B_s^{-2} - B_e^{-2})c^2 \right]^{-1/2}, \quad (24)$$

$$B_c^0 = \left[B_{ec}^2 + (B_s^2 - B_{ec}^2)c^2 \right]^{1/2}. \quad (25)$$

The tensor assembled by the source is equivalently

$$B^0 = B_{l0} \mathbf{t} \otimes \mathbf{t} + B_c^0 \mathbf{n} \otimes \mathbf{n} + B_g^0 \mathbf{m} \otimes \mathbf{m}. \quad (26)$$

At pure edge character, $c^2 = 0$, this reduces to line, climb, and edge drags B_{l0}, B_{ec}, B_e along $\mathbf{t}, \mathbf{n}, \mathbf{m}$.

6 Hybrid residual and Jacobian

For a regular arm i , the drag-force contribution assigned to the current node is

$$\mathbf{R}_i^{\text{drag}} = \frac{1}{2}(1 - \phi_i) [L_{e,i} D_{e,i} \mathbf{v} + L_{s,i} \mathbf{F}_s(\mathbf{v}; \mathbf{b}_i)] + \frac{1}{2} \phi_i L_i B_i^0 \mathbf{v}. \quad (27)$$

For a product-zero/junction arm it is

$$\mathbf{R}_i^{\text{drag}} = \frac{1}{2} L_i \left[(1 - \phi_i) D_{j,i} + \phi_i B_i^0 \right] \mathbf{v}. \quad (28)$$

Notice that the BCC_0 part owns the full physical arm length L_i ; it is not decomposed into L_s and L_e .

Collinear screw Burgers vectors are grouped. Define

$$C_g = \frac{1}{2} \sum_{i \in g} (1 - \phi_i) L_{s,i} \quad (29)$$

and let D_{lin} contain all edge/junction terms plus all inside BCC_0 tensors. The Newton solve uses the force residual

$$\mathbf{r}(\mathbf{v}) = \mathbf{F}_{\text{node}} - D_{\text{lin}} \mathbf{v} - \sum_g C_g \mathbf{F}_s(\mathbf{v}; \mathbf{b}_g) = \mathbf{0}, \quad (30)$$

with Jacobian

$$J(\mathbf{v}) = -D_{\text{lin}} - \sum_g C_g D_s(\mathbf{v}; \mathbf{b}_g). \quad (31)$$

The iteration applies $\mathbf{v} \leftarrow \mathbf{v} - J^{-1} \mathbf{r}$, using the convergence and correction controls inherited from BCC_nl.

The original BCC_nl one-screw shortcut solves only a two-dimensional nonlinear subspace and restores the Burgers-direction velocity analytically. A mixed BCC_0 tensor need not have $\mathbf{b}$ as an eigenvector. Therefore the new implementation disables that shortcut whenever any arm has $\phi_i > 0$, and uses the full three-dimensional Newton system instead.

7 Frames, outputs, and limiting cases

Intersection geometry is evaluated in the simulation/laboratory frame: the minimum-image arm $\mathbf{r}_{\text{lab}}$, node position, and nearest periodic inclusion center are passed to the ellipsoid routine. A saved copy of $\mathbf{r}_{\text{lab}}$ isolates this calculation from crystallographic rotation.

When `useLabFrame` is enabled, force, velocity, arm direction, and Burgers vector are rotated with `rotMatrixInverse`; all drag tensors and the Newton solve are then evaluated in the crystal frame. The converged velocity is rotated back with `rotMatrix`. For an orthogonal rotation Q , this is the covariance relation

$$B_{\text{crystal}}^0 = Q^T B_{\text{lab}}^0 Q. \quad (32)$$

The important limits are:

$$\phi_i = 0 \ \forall i \implies \text{the copied BCC_nl algebra and execution path,} \quad (33)$$

$$\phi_i = 1 \ \forall i \implies D_{\text{lin}} = \sum_i \frac{1}{2} L_i B_i^0, \quad C_g = 0. \quad (34)$$

Thus complete coverage gives the same nodal velocity as `Mobility_BCC_0` when `includeInertia=0`. The nonlinear parent law does not contain BCC_0's optional mass term, so inertial parity is not claimed when `includeInertia=1`.

The `mobArgs->invDragMatrix` output intentionally follows the inherited BCC_nl convention: it is an inverse of the negative Newton Jacobian in the solver frame. Even at complete coverage, it is not the positive lab-frame B^{-1} stored by `Mobility_BCC_0`; the nodal velocity is nevertheless the BCC_0 velocity under the non-inertial condition above.

8 Parameters and control-file use

Parameter group	Used by	Meaning
<code>inclusionFile</code>	Geometry	Inclusion centers, radii, orientations, and candidate intersection data.
<code>MobScrew</code> , <code>MobEdge</code> , <code>MobClimb</code>	BCC_0 inside	Mobilities whose reciprocals define Equation (19).
<code>MobGlide</code> , <code>MobLine</code> , <code>MobClimb</code>	BCC_nl outside	Linear edge, line, and climb drags.
<code>MobPeierls</code> , <code>MobExpP</code> , <code>MobExpQ</code> , <code>MobAlpha</code> , <code>MobDeltaH0</code>	BCC_nl outside	Thermally activated screw law.
<code>MobCoeffA</code> , <code>MobCoeffC0</code> , <code>MobCoeffC</code> , <code>MobCoeffD</code> , <code>meltTemp</code> , <code>TempK</code> , <code>pressure</code>	BCC_nl outside	Initializer constants in Equations (7)–(9).
<code>MobFileNL</code> , <code>ecFile</code> , <code>meltTempFile</code>	Existing nonlinear initialization paths	Optional table/file inputs retained consistently with BCC_nl.
<code>MobEshelbyResist</code>	Not used	Deliberately disabled for this selector; the inside branch is the literal BCC_0 tensor, not a scalar-rescaled outside law.

The following is a ready-to-use control-file example based on `tests/screwEsh.ctrl`. It includes the simulation controls needed by that test as well as the complete parameter union used by the hybrid law. The material constants shown are the existing tantalum defaults in `src/Param.cc`; they should be replaced by the calibration for the material being simulated.

8.1 Complete example control input

```
dirname = "tests/screwEsh_nl_Eshelby_results"
inclusionFile = "tests/screwEsh.dat"

# Domain and cell geometry
numXdoms = 1
numYdoms = 1
numZdoms = 1
numXcells = 3
numYcells = 3
numZcells = 3

# Discretization and topological changes
maxSeg = 50
minSeg = 15
remeshRule = 2
enforceGlidePlanes = 0
enableCrossSlip = 0
splitMultiNodeFreq = 5

# Time integration
maxstep = 1000000
timestepIntegrator = "trapezoid"
nextDT = 1.0e-12
rTol = 0.5

# Fast multipole controls
fmEnabled = 0
fmMPOrder = 2
fmTaylorOrder = 5

# Loading and thermodynamic state
loadType = 1
eRate = 1.0e5
indxErate = 2
edotdir = [
    0.0
    0.0
    1.0
]
TempK = 600.0
pressure = 0.0

# Select the hybrid law
mobilityLaw = "BCC_nl_Eshelby"
materialTypeName = "BCC"
```

```

# Elastic and core parameters used by the simulation
shearModulus = 8.600000e10
pois = 2.910000e-1
burgMag = 2.480000e-10
YoungModulus = 2.220520e11
rc = 2.0
rann = 5.0
collisionMethod = 3

# BCC_0 mobility used on inclusion-covered arm portions
MobScrew = 1.0e2
MobEdge = 1.0e2
MobClimb = 1.0e-5

# BCC_nl mobility used on uncovered arm portions
MobGlide = 1.0e2
MobLine = 1.0e4
MobPeierls = 3.240000e8
MobDeltaH0 = 1.728748e-19
MobAlpha = 9.661236e-1
MobExpP = 0.5
MobExpQ = 1.23
MobCoeffA = 1.525
MobCoeffC = 2.032370e-1
MobCoeffC0 = 2.048000e3
MobCoeffD = 1.626950e-4
meltTemp = 3.170683e3

# Optional nonlinear-property files; empty uses values above
MobFileNL = ""
ecFile = ""
meltTempFile = ""

# Output controls
fluxfile = 1
fluxfreq = 200
savecn = 1
savecnfreq = 200
saveprop = 1
savepropfreq = 200
savetimers = 0
velfile = 0
velfilefreq = 200

```

The associated nodal data and particle file for this example are `tests/screwEsh.data` and `tests/screwEsh.dat`. A one-step serial smoke run can be launched as

```

bin/paradis -maxstep 1 -s -d tests/screwEsh.data \
  path/to/BCC_nl_Eshelby.ctrl

```


8.2 Inputs specific to the hybrid switch

Only three changes are required when converting an existing `BCC_nl` control file:

1. set `mobilityLaw = "BCC_nl_Eshelby"`;
2. provide a nonempty `inclusionFile`;
3. provide positive `MobScrew` and `MobEdge` values for the `BCC_0` inside tensor. `MobClimb` is already required by `BCC_nl` and is shared by both branches.

`MobEshelbyResist` is not used by this selector and should be omitted.

An empty `inclusionFile` is permitted; it makes the law reduce to `BCC_nl` everywhere.

9 Build and verification

The selector, prototypes, and implementation are guarded by `ESHELBY`. The object remains in the common source list, like the other Eshelby mobility objects; without `ESHELBY` it compiles to an empty translation unit and the selector is absent. No changes are required in `Param.cc`, `Param.h`, or `Initialize.cc`, because mobility selection and function-pointer initialization are generic.

The implementation was checked in the following ways:

- full serial builds with and without `ESHELBY`;
- a one-step ParaDiS run using `BCC_nl_Eshelby` and an inclusion file;
- symbolic comparison of every `BCC_0` tensor branch with `MobilityLaw_BCC_0.cc`;
- 10^5 randomized orientations over wide mobility ranges, with only floating-point ordering differences (approximately 10^{-12} at deliberately extreme ranges and near machine precision ordinarily);
- direct checks of overlapping, tangent, disjoint, and fully contained inclusion intervals;
- algebraic checks of the $\phi = 0$ and $\phi = 1$ limits.

10 Implementation cautions

1. “Inside” is arm- and endpoint-weighted through Equation (6); a node center inside an inclusion does not imply that all its arms have $\phi = 1$.
2. The three `BCC_0` input mobilities must be finite and positive because the source uses their reciprocals.
3. Complete `BCC_0` equivalence assumes `includeInertia=0`.
4. `MobEshelbyResist` has no effect on this law by design.
5. The outside-only branch deliberately retains the numerical behavior of the original `BCC_nl` implementation. Baseline solver changes should be made separately in both laws if that behavior is later revised.